Metodologia del corso di Informatica

Versione 1.5 — 13/11/2026 — Prof. Nicola Regge, Laboratorio di Informatica, Piamarta Milano.

Documento del docente (in italiano, senza nomi di allievi). Raccoglie in un solo posto il metodo che usiamo, la giornata di lezione, i compiti, la valutazione, la chiusura delle classi, la documentazione per la Regione, l'automazione, i comandi da dare a Claude e dove stanno i documenti. Le regole complete e datate sono in `RIFERIMENTI-E-DECISIONI.md`; questo documento ne è la sintesi operativa da stampare.

---

00 Come si usa questo documento

1. È la **sintesi da tenere sul banco**: per ogni situazione dice cosa si fa e quale comando si dà a Claude.
2. La fonte di verità delle regole resta `RIFERIMENTI-E-DECISIONI.md` (versione 2.23): il numero tra parentesi quadre, per esempio [§2.39], rimanda alla regola completa.
3. Quando una regola cambia, si aggiorna prima `RIFERIMENTI-E-DECISIONI.md`, poi questo documento con una versione nuova. La versione stampata resta congelata.

> **Approvato e da provare.** In classe si usa solo ciò che il docente ha provato e approvato [§2.41]. Ciò che è pronto ma non ancora provato è segnato **DA PROVARE** in questo documento (riepilogo al capitolo 13).

01 Missione e contesto

1. **Chi sono i ragazzi.** Molti hanno un background migratorio, a volte contesti difficili, spesso arrivano "scartati" da altre scuole. Qui non sono scarti: ricevono una cosa fatta bene. Il tono è sempre di rispetto e fiducia.
2. **Il patto di fiducia.** I ragazzi stimano molto il docente perché è lui a dargli questa possibilità. È la leva più forte e insieme la responsabilità di non deluderli: qualità alta, sempre.
3. **La missione.** Un percorso di qualità superiore, con dignità, per sbocchi lavorativi migliori. Non è un corso di serie B: livello alto, modo accessibile.
4. **Il vincolo della scuola.** Niente installazioni (servirebbe l'amministratore di sistema): si usano browser, programmi portabili e ciò che c'è già (Lazarus, Blocco note, Google, GitHub da browser).

02 Il motore del coinvolgimento

1. **Il problema numero 1** non è che non capiscano: è che si stufino e mollino, per difesa. Il compito è far sentire che provarci conviene e non fa male.
2. **Vinci subito · Fallo tuo · Mostralo**, sempre tutte e tre:
    1. **Vinci subito** (circa 10 minuti): il primo risultato è quasi impossibile da sbagliare.
    2. **Fallo tuo**: in ogni esercizio scelgono qualcosa di loro (frase, colori, nome, passioni).
    3. **Mostralo**: ogni pezzo è giocabile e mostrabile al compagno o sul telefono.
3. **Passi minuscoli con "FATTO" visibile**: un obiettivo alla volta, una piccola vittoria a ogni passo; niente tre ore di fila sullo stesso compito [§2.7].
4. **Errore senza vergogna**: il bug è normale, succede anche ai professionisti.

5. **La prova del nove**: se sanno spiegare a voce, con parole loro, cosa hanno fatto, la competenza c'è. Vale anche per l'uso dell'intelligenza artificiale (IA): aiuta a capire, non a saltare il pensiero.
6. **Sovradimensionare e gestire il divario** [§2.24]: un nucleo base che tutti riescono a fare e consegnare, più sfide per chi finisce prima, così nessuno resta fermo.
7. **Carta e penna in ogni lezione (tassativo)**: ogni allievo ha carta e penna sul banco per appunti e schemi a mano; chi non li ha li riceve dal docente, che segna una nota.
8. **Solo il metodo del docente** [§2.26]: se i ragazzi faticano si spiega lo stesso metodo in modo più semplice, mai un metodo alternativo.

03 Come si scrive per i ragazzi

1. **Un'azione per passo**, numerata. Mai due clic nello stesso passo.
2. **Prima il dove, poi il cosa**: il programma (Lazarus, Windows, GitHub, Classroom), la zona dello schermo, il nome del bottone, l'azione.
3. **"Adesso devi vedere"** sotto ogni passo, così il ragazzo sa se l'ha fatto bene. **"Non mi torna"** con il rimedio all'errore tipico di quel passo.
4. **Niente da scrivere a mano**: nomi e codice si copiano con il bottone giallo e si incollano con Ctrl + V [§2.20].
5. **Mai riconoscere un file dall'estensione**: sui computer della scuola le estensioni sono nascoste. Si fa scegliere tutta la cartella (Ctrl + A).
6. **Lingue** [§2.4]: italiano per tutti; arabo e cinese semplificato per la Classe 1; bangla solo per chi serve in Classe 3. Indirizzi ed email non si traducono; i bottoni dei siti si descrivono per posizione e colore, non con il nome inglese.
7. **Stesse cose, stesso layout** [§2.19]; codice Lazarus con i rientri allineati (if con else, begin con end) [§2.27].
8. **Testi su Classroom già impaginati** [§2.29]: titoletti in maiuscolo, una riga per passo, il link su una riga sua.
9. **Una pagina per i ragazzi e un link per il docente**: tutto il materiale del giorno sta in una sola pagina del sito del corso; il docente ha la sua pagina di controllo.

04 La giornata di lezione

04.1 Prima della lezione

1. **Report dello svolto** della classe, poi 3-4 argomenti proposti; il docente sceglie [§2.16, §2.23].
2. **Scheda sintetica** di una pagina per la lezione frontale alla lavagna [§2.22].
3. Claude prepara **pagina dei ragazzi, compito su Classroom (in coda, fermo) e testi del registro** nel piano della settimana (`orario/piano-settimana-...`).
4. I compiti restano in coda nello stato **in attesa dell'OK del docente**: non partono da soli.

04.2 In classe

1. **Firma delle ore subito, entro le 14:05** (vincolante): si firma il programma previsto, anche provvisorio; le ore non firmate vengono tolte dal monte ore della Regione. Testo del registro corto, massimo circa 40 caratteri, uno per ora [§2.25].
2. **Presenze**: il docente le manda a Claude; gli assenti si escludono da liste, pagine e conteggi [§2.35].

3. **Carta e penna sul banco** (e il materiale del giorno, per esempio cacciaviti e forbici quando servono).
4. **OK del docente** ("OK 1INF") e lo script pubblica il compito su Classroom entro 5 minuti.
5. **Un solo compito aperto alla volta** [§2.39]: il secondo si pubblica quando il docente dice che la classe ha finito il primo. Titoli con l'ordine ("1 di 2").
6. **Solo il lavoro del giorno** su Classroom e sulle pagine [§2.38].
7. **Risposte cortissime di Claude in classe** [§2.33]: il testo del compito da incollare e il link del docente, niente altro se non richiesto.
8. **Monitoraggio con Veyon**: dalla miniatura solo fatti ("PC 12 su Google Maps alle 11:30"), mai giudizi [§2.32]; il fuori compito si annota sempre [§2.36].

04.3 A fine lezione

1. La raccolta delle consegne è automatica: alla scadenza più 10 minuti.
2. **Chiudi classe** solo dopo l'ultima raccolta (capitolo 07).
3. Claude chiede gli **argomenti realmente svolti** e aggiorna `ARGOMENTI-SVOLTI-2026-27.md`.

05 Il compito: dal testo alla consegna

1. **Struttura**: pagina unica dei ragazzi (spiegazioni, passi, materiali) + compito su Classroom con il testo impaginato + Documento Google da compilare.
2. **Riflessione obbligatoria** in ogni compito [§2.2]: "cosa NON ho fatto e perché" e "cosa NON ho capito". È la parte più importante da leggere.
3. **Un solo Documento** per la consegna [§2.6] (screenshot, codice, spiegazione, riflessione), non file sparsi. 3bis. **Che cosa si consegna (decisione del docente, 13/11/2026):** di norma un Documento (con gli screenshot di ciò che l'allievo ha fatto, tutti in UN solo Documento) e/o un quiz su Moduli; a volte entrambi, a volte uno dei due, raramente altro.
4. **Lavori su GitHub**: caricamento con Add file, poi Upload files, poi la cartella del progetto con Ctrl + A. Il ragazzo su Classroom preme Consegna; il codice lo legge Claude dal repository [§2.34].
5. **Controllo dei lavori su GitHub** [§2.40]: i repository si trovano con la ricerca di GitHub (creati oggi), i file si leggono scaricandoli (clone). Mai dire "non ha caricato" senza aver controllato.
6. **Documenti già su Classroom sono congelati** [§2.11]: si correggono solo se l'errore è grave, avvisando la classe; mai cambiare il metodo a lezione iniziata.

06 La valutazione

1. **Voti in centesimi**: O = oggettivo (dalla griglia), C = commisurato (per chi ha un percorso personalizzato).
2. **Griglia scritta prima**, con i punti per voce; le penalità sono dichiarate (per esempio fuori compito osservato: da −5 a −15 centesimi).
3. **Valutazione dell'AI** [§2.15]: Claude propone voto e giudizio, sempre marcati come "dell'AI"; il docente conferma, corregge o sovrascrive.
4. **Prova del nove orale**: tre domande per verificare che il ragazzo sappia spiegare.
5. **Recupero a fianco** [§2.42]: chi ha meno di 60 o non ha svolto rifà l'esercizio in versione semplificata con il docente accanto (pagine `recupero/` del sito; elenco con i nomi nel repository riservato). Proposta da confermare: voto nuovo sul registro, il precedente resta; se non sa rispondere alle domande non supera 60.

6. **Voti su Classroom in bozza (DA PROVARE)**: lo script li scrive come bozza nei compiti creati dallo script; il docente li controlla e preme "Restituisci".
7. **Valutazione complessiva della classe** [§2.43]: il punto su tutti i lavori fino a una data, con la lista standard per il docente; i tre con la media più bassa vengono interrogati per recuperare.
8. **Voti su DIDAweb**: si inseriscono a mano, dall'elenco già in ordine alfabetico con la descrizione corta (`archivio-prove/didaweb/`). Da verificare se DIDAweb permette di caricarli da file.

07 Chiudi classe

Il comando **"Chiudi classe 2INF"** chiude il lavoro della classe [§2.30]. Prima Claude dice cosa genera e aspetta il **"via"**.

1. **Per ogni allievo presente**: il libro individuale (teoria dal primo giorno, lavori, voti, cosa fare), in versione protetta da password per il ragazzo e senza password per il docente.
2. **Per il docente**: documento di chiusura (griglie, voti per il registro, report, monitoraggio, fuori compito, cosa non hanno capito), foglio Excel della classe aggiornato.
3. **Due zip separati** [§2.14]: DOCENTE senza password; LIBRI PROTETTI per i ragazzi. Mai documenti del docente nello stesso pacchetto dei ragazzi.
4. **Assenti**: il loro libro va nella cartella `assenti`, fuori dagli zip.
5. Tutto con i nomi resta solo nel **repository riservato** (`dati/chiusure/<CLASSE>/`).
6. Se un file supera il limite di invio (30 MB), si divide prima e si dice esattamente quali file scaricare.

08 Documentazione per la Regione

1. **Firma delle ore** entro le 14:05, ogni giorno.
2. **Argomenti svolti** (`ARGOMENTI-SVOLTI-2026-27.md`), agganciati all'Allegato A.
3. **Allegato A** (programma per competenze): le parti del docente in `allegato-a-2026-27/`.
4. **Archivio delle prove (DA PROVARE)** nel repository riservato (`archivio-prove/<CLASSE>/`): per ogni prova la scheda (data, classe, competenza, testo del compito, passi, griglia, presenze, esiti e voti per allievo), la pagina del compito fotografata quel giorno e la chiusura; per ogni classe il **registro delle prove** in PDF.
5. **Consegne su Classroom**: restano sul corso Classroom insieme ai voti; per i lavori su GitHub la prova è nel repository dell'allievo e nell'archivio.

09 L'automazione: Git, script e Classroom

1. **Due repository** [§2.5]:
    1. **pubblico** `corso-informatica`: materiali, pagine del sito (cartella `docs/`), mai nomi di minori;
    2. **riservato** `corso-informatica-riservato`: nomi, voti, presenze, consegne, chiusure, coda dei compiti, configurazioni.
2. **La coda dei compiti** (`coda/*.json` nel riservato): un file per compito, con stato:
    1. **in attesa dell'OK del docente**: preparato, fermo;
    2. **da pubblicare**: dopo l'OK, lo script lo pubblica;
    3. **pubblicato**, poi **chiuso** dopo la raccolta.

3. **Lo script di Classroom** (`strumenti/automazione-corso.gs`, progetto "Consegne Classroom → Git" sull'account della scuola) gira da solo ogni minuto (dal 12/10, versione 7.1; se il giro prima non ha finito, salta) e lavora nei due sensi:
    1. da Git a Classroom: pubblica i compiti (solo quelli con la data di oggi); nella versione 3, DA PROVARE, aggiorna i testi e scrive i voti in bozza;
    2. da Classroom a Git: alla scadenza porta le consegne nel repository riservato.
4. **Limiti**:
    1. i compiti creati a mano su Classroom non si possono modificare né votare dallo script, perciò i compiti li crea sempre lo script;
    2. ogni nuova versione dello script la incolla il docente una volta (mai subito prima delle lezioni);
    3. DIDAweb non ha un accesso per i programmi: niente automazione.
5. **Pagine di controllo del docente**: leggono i siti dei ragazzi direttamente, senza le API di GitHub (che si bloccano dalla rete della scuola) [§2.37].

09b Il giro delle Pull Request (approvato il 12/10/2026)

1. **Una sola pagina, un solo compito** su Classroom, con un link fisso che si aggiorna da solo [REGOLE-COSTRUZIONE-TEORIA].
    1. FASE 1: la teoria (il Documento del ragazzo è un ramo dell'albero delle versioni; come si chiede una modifica; lo script che gira).
    2. FASE 2: due bottoni verdi, in ordine: «1) Feedback» (il parere sulla teoria) e «2) PR1» (il Modulo «Consegna»).
    3. La dispensa Agile è a parte (Manifesto Agile e 10 regole), con il suo ramo nel grafo.
2. **La Pull Request (PR)** si chiama PR1, PR2, PR3… e ha 4 parti: Titolo · Cosa cambia · Dove · Perché. Si manda dal Modulo «Consegna», generico per tutte le classi, con la casella di testo grande.
3. **Il backlog** (una pagina per classe: `docs/3inf-backlog/`, `docs/2inf-backlog/`):
    1. colonne: Posizione, Data e ora, Alunno (nome e iniziale), PR, Riepilogo della richiesta, Peso, Perché, Stato;
    2. ogni richiesta vale 1, chiunque la faccia; ordine: peso più basso prima (meno lavoro sul codice), a parità di peso chi è arrivato prima;
    3. peso con i numeri di Fibonacci: 1 testo o colore · 2 un numero · 3 un personaggio nuovo · 5 ambiente o oggetto nuovo · 8 regola nuova del gioco · 13 parte grande nuova;
    4. il riepilogo lo scrive Claude leggendo il testo e i file della PR;
    5. Stato: FATTO in verde con la versione che l'ha portata nel gioco, oppure NO in rosso con il motivo (es. foto di una persona vera).
4. **Il giro di Claude, per ogni PR nuova** (entro pochi minuti):
    1. la legge (testo e file allegati, esportati in automatico nel repository riservato);
    2. scrive riepilogo, peso e motivo nel backlog;
    3. la mette in produzione come versione nuova del gioco della classe (una PR = una versione, numero minor: v0.12, v0.13…), con la cartella ferma della versione;
    4. scrive il changelog (cosa cambia, chi l'ha chiesta, numero della PR);
    5. il grafo delle versioni si aggiorna da solo (legge `versioni.json`);
    6. segna la PR come FATTO nel backlog con il numero della versione;
    7. controlla i link: versione giocabile, changelog, grafo, backlog.

5. **Tempi**: lo script raccoglie le risposte ogni minuto e subito a ogni invio dei Moduli collegati; il backlog si rigenera da solo.
6. **Privacy**: nessuna foto di persone vere nel gioco; nel backlog solo nome e iniziale.
7. **Come si rifà**: la guida `GIRO-PULL-REQUEST.md` (PDF `20261012_Giro-Pull-Request_Docente_v1.0.pdf`): un file di impostazioni per lezione (`config/lezioni/<id>.json` nel riservato), il giro automatico `generatori/lezione_pr.py` (un solo processo scrive su Git), la messa in produzione `strumenti/rilascia_versione_pr.py`, la pagina «Cosa è cambiato oggi» `strumenti/gen_novita.py`; voti con le regole del docente e valutazione per allievo in PDF.
8. **Lingue**: ogni classe ha le sue (1INF IT/AR/ZH, 2INF SOLO italiano, 3INF IT/BN); per le classi solo italiano ogni pagina passa da `strumenti/solo_italiano.py`, che blocca la pubblicazione se resta un'altra lingua.

10 I comandi da dare a Claude

Comando	Quando	Cosa succede
PPP (o "parcheggia")	quando si vuole annotare senza riempire la chat	in coda, nessun output (né in chat né nelle finestre visibili): Claude lavora con un agente in background; consegna dopo "avanti" (CLAUDE.md R3)
PRI	quando una cosa deve passare davanti	diventa la prima cosa da fare (CLAUDE.md R3); PRI+ non si usa più dal 04/11/2026
PUBB n (n da 1 a 6)	quando si vuole pubblicare il lavoro di un'ora	Claude pubblica su Classroom il lavoro dell'ora n (1 = 8-9 … 6 = 13-14); se non risulta PUBLISHED, prima riga «PUBB n NON riuscito: pubblica a mano» con il pacchetto (CLAUDE.md R2)
avanti	dopo uno o più PPP	Claude consegna quello che era parcheggiato
via	dopo «aspetta il via» o dopo che Claude ha detto cosa farà	Claude esegue; prima del «via» risponde solo «Annotato» (CLAUDE.md R3)
Valutazione complessiva 1INF	quando si vuole fare il punto su tutti i lavori	Valutazione complessiva della classe: libri protetti, voti O e C per il registro, griglia, situazione, recupero, due zip
Chiudi classe 2INF	a fine lavoro, dopo l'ultima raccolta	libri individuali, zip docente e ragazzi, griglie, voti, report
monito, o uno screenshot di Veyon senza testo	durante la lezione	Claude annota i fatti nel report di andamento (riservato)
chi manca?	durante la lezione	elenco di chi non ha ancora fatto o consegnato (assenti esclusi)
dammi il link per controllare	durante la lezione	link della pagina di controllo del docente
dammi il testo per Classroom	quando serve	testo del compito impaginato, in un solo blocco da copiare
dammi da scaricare qui	quando si vogliono i file	Claude manda i file in chat (divisi se troppo grandi)
ricordami …	quando serve	Claude programma un promemoria che arriva in questa chat all'ora giusta
prepara lezioni per domani	il giorno prima	materiali, compiti in coda, testi del registro, promemoria

Comando	Quando	Cosa succede
FIRMA · SVOLTO · VOLO · LAVAGNA · STATO	dall'Atlante	firma ore, argomenti svolti, voto "domanda al volo" (70 OK / 50 KO), foto della lavagna nei libri, stato dei dati

11 I documenti: dove sta cosa

11.1 Repository pubblico (corso-informatica)

1. `RIFERIMENTI-E-DECISIONI.md`: tutte le regole datate (fonte di verità).
2. `REGISTRO-ERRORI-CLAUDE.md`: ogni errore segnalato dal docente, con la regola nata da lì [§2.28].
3. `METODOLOGIA-CORSO.md`: questo documento.
4. `ATLANTE.md`: la mappa dei libri, dei tipi di file e dei comandi.
5. `CLAUDE.md`: istruzioni per Claude e indice di tutti i documenti con le versioni.
6. `orario/`: orario e piano della settimana, con i testi del registro.
7. `ARGOMENTI-SVOLTI-2026-27.md`, `allegato-a-2026-27/`: documentazione per la Regione.
8. `docs/`: le pagine del sito per i ragazzi (una per classe e argomento), il motore dei quiz, le pagine di recupero.
9. `classe-1/` … `classe-4/`: materiali delle classi (dispense, compiti, generatori).
10. `strumenti/`: generatori comuni (pagine di recupero, nomi dei file).

11.2 Repository riservato (corso-informatica-riservato)

1. `coda/`: i compiti per lo script di Classroom.
2. `strumenti/automazione-corso.gs`: lo script di Classroom.
3. `config/`: classi e corsi Classroom, utenti GitHub dei ragazzi, elenco delle prove.
4. `dati/presenze/`, `dati/andamento/`, `dati/consegne/`, `dati/chiusure/`, `dati/recupero/`: presenze, monitoraggio, consegne, chiusure, recuperi.
5. `archivio-prove/`: documentazione delle prove per la Regione e voti pronti per DIDAweb.
6. `generatori/`: chiusure delle classi, archivio delle prove, recupero.
7. `strategico/`: password dei libri (mai incollate in chat).

12 Regole di lavoro di Claude

1. **Ogni correzione del docente diventa regola** e va nel registro degli errori [§2.28].
2. **Su Classroom si pubblica solo con il comando PUBB n** (dal 04/11/2026, CLAUDE.md R2), un'ora per volta; il pacchetto per il post a mano è sempre pronto.
3. **Verificare prima di affermare**: mai dire "non ha fatto" o "non ha caricato" senza aver controllato.
4. **Tutto ciò che il docente deve copiare** sta in un blocco con il bottone copia, una cosa per blocco, subito dopo il passo; ogni passo è una frase con il verbo e il «dove» detto a parole (CLAUDE.md R4).
5. **Privacy**: nomi, voti e presenze dei minori solo nel repository riservato; le password non si scrivono mai in chat.
6. **Solo strumenti approvati** in classe; ciò che è nuovo si prova prima con il docente [§2.41].
7. **File grandi**: si dividono prima di mandarli e si dice esattamente quali scaricare.

13 Stato: approvato e da provare (al 06/10/2026)

Approvato (provato in classe):

1. pubblicazione del compito con lo script;
2. raccolta automatica delle consegne;
3. una pagina per i ragazzi e un link per il docente;
4. caricamento su GitHub con Upload files e Ctrl + A;
5. Chiudi classe con libri individuali protetti, zip del docente e voti in centesimi.

Da provare con il docente:

1. voti in bozza su Classroom (script versione 3);
2. archivio delle prove e registro delle prove;
3. recupero a fianco (pagine semplificate ed elenco del docente);
4. pagina del compito unica in 3 versioni (normale, semplificata, lingue) e correttore unico: da costruire.

14 Glossario delle sigle

Sigla o termine	Significato
IA / AI	intelligenza artificiale
PDF	formato di documento da leggere e stampare
MD	Markdown, il testo sorgente dei documenti (lo legge anche l'IA dei ragazzi)
HTML / CSS	linguaggi delle pagine web (struttura e aspetto)
PFP	piano formativo personalizzato
Allegato A	il programma per competenze richiesto dalla Regione
Repository	cartella di progetto su GitHub, con la storia delle modifiche
Commit	salvataggio di una modifica su GitHub
Veyon	programma per vedere gli schermi dei computer del laboratorio
DIDAweb	il registro elettronico della scuola
O / C	voto oggettivo / commisurato

15 Registro delle versioni

6. **v1.5 (13/11/2026)**: capitolo 05, punto 3bis, la consegna è di norma un Documento e/o un quiz su Moduli (decisione del docente).
7. **v1.4 (04/11/2026)**: capitolo 10 con i comandi PUBB n, PRI e «via» (tolti PRI+ e «OK 1INF»); capitolo 12 punti 2 e 4 allineati a CLAUDE.md R2 e R4; capitolo 16 con il testo spostato da CLAUDE.md (missione, motore, contesto didattico, metodo, gruppi, flusso Git, lezione e cruscotto).
8. **v1.3 (12/10/2026)**: 09b punto 8, come si rifà il giro delle Pull Request (guida e strumenti).
9. **v1.2 (12/10/2026)**: capitolo 09b, il giro delle Pull Request con backlog, pesi, versioni e grafo; script ogni minuto.
10. **v1.1 (06/10/2026)**: aggiunta la Valutazione complessiva della classe (comando e capitolo 06).

11. **v1.0 (06/10/2026)**: prima versione, dalla sintesi di `RIFERIMENTI-E-DECISIONI.md` v2.23, dell'Atlante e del lavoro di settembre-ottobre 2026.

16 Testo spostato da CLAUDE.md il 04/11/2026

> **Testo spostato da `CLAUDE.md` il 04/11/2026, senza cambiarlo** (archivio completo: `archivio/CLAUDE-fino-al-2026-11-04.md`). Dove questo testo dice una cosa diversa dalle regole R1-R11 del nuovo `CLAUDE.md` (per esempio prefissi, pubblicazione, tabelle di passi, carta e penna), valgono le regole R1-R11.

16.1 Missione, motore del coinvolgimento e autoregolazione

Contesto umano e missione (leggere SEMPRE per prime)

Gli studenti vivono spesso situazioni di **svantaggio sociale**: molti hanno un **background migratorio** (extracomunitari), a volte contesti familiari difficili, e non hanno avuto molte opportunità. Spesso sono ragazzi **"scartati" da altre scuole** (respinti o messi da parte da altri istituti) e arrivano qui già segnati da quei rifiuti. Alcuni possono essere "difficili" da coinvolgere e tenere agganciati.

> Proprio per questo il modo in cui li trattiamo conta doppio: qui **non sono scarti**, sono ragazzi a cui diamo una cosa fatta bene. Il tono è sempre di **rispetto e fiducia**, mai di sufficienza.

Il legame di fiducia (leva + responsabilità): questi ragazzi hanno **grande stima nel docente**, proprio perché è lui a dargli questa possibilità. È il motore motivazionale più forte che abbiamo — e insieme la responsabilità di **non deluderli**: mantenere alta la qualità e non far mai sentire quel patto tradito.

La missione del docente (Nicola): offrirgli un percorso di **qualità superiore**, con **dignità**, perché possano trovare **sbocchi lavorativi migliori**. Questo NON è un corso "di serie B": è pensato apposta per **aprire porte**. Va tenuto alto il livello *e* accessibile il modo.

Implicazioni pratiche (VINCOLANTI): - **Approccio molto semplice, tutto VISUALE.** Niente riga di comando — né per Git né per altro. Si **clicca**, non si digitano comandi. - Per **Git**: solo strumenti visuali. Consigliato **GitHub Desktop** (il più semplice, bottoni grandi e chiari) oppure il **pannello Git di VS Code**. **MAI la CLI** con loro. - Passi **piccolissimi**, un obiettivo alla volta, sempre con un risultato concreto e "figo" da vedere subito. - **Zero gergo** non spiegato; ogni termine tecnico/inglese va tradotto. - **Tanto incoraggiamento**: celebrare ogni piccola vittoria; non far mai sentire "stupido" nessuno. La pazienza è parte del metodo, non un extra. - Risultati **spendibili e mostrabili** (giochi che funzionano, cose di cui essere fieri): sono la leva motivazionale più forte per questi ragazzi.

Il motore del coinvolgimento (il problema numero 1)

Il rischio vero non è che non capiscano: è che **si stufino e mollino**. Molti hanno imparato che "provarci = fallire ed essere umiliati", quindi si arrendono *prima*, per difesa. Il compito numero 1 è **far sentire che qui provarci conviene e non fa male**. Ogni esercizio si progetta attorno a questo motore:

"Vinci subito · Fallo tuo · Mostralo" — sempre tutte e tre: 1. **Vinci subito (≈10 min):** il primo risultato deve essere quasi impossibile da sbagliare. Poche mosse → qualcosa che funziona a schermo. La prima vittoria è ciò che impedisce la fuga. 2. **Fallo tuo:** in ogni esercizio scelgono qualcosa di **loro** (colore, nome del gioco, personaggio, foto/meme, scritta).

L'ownership è il gancio più forte. 3. **Mostralo:** ogni pezzo dev'essere **giocabile e mostrabile** (al compagno, sul telefono). Il "l'ho fatto io" motiva più di un voto.

Due ingranaggi di supporto: - **Passi minuscoli con "FATTO!" visibile:** mai più di un obiettivo alla volta; ognuno chiude con una piccola vittoria concreta. - **Errore = zero vergogna:** annulla facile, il bug è normale ("succede a tutti i programmatori, anche ai pro"). Nessuno si sente stupido, mai. - **La prova del nove — "saperlo spiegare":** se sanno raccontare a voce, con parole loro, cosa fa ciò che hanno fatto (anche se in parte copiato), la competenza c'è davvero. È anche la regola d'uso dell'AI: aiuta a capire, non a saltare il pensiero.

AUTOREGOLAZIONE (13/10/2026, parola da ricordare): la leva non è il voto ma l'autodeterminazione. Ogni pagina per i ragazzi pubblica SEMPRE gli strumenti per regolarsi da soli (monitor della lezione, il proprio andamento, risultati); ogni schermata Veyon va subito nel monitor pubblico. Il docente ha in più il suo cruscotto cifrato. Vedi RIFERIMENTI §2.52, REGISTRO-ERRORI #60.

> Conseguenza pratica sugli esercizi: si parte SEMPRE da qualcosa di **giocabile e personalizzabile**, non da esercizi "scolastici" astratti.

16.2 Contesto didattico, obiettivo, metodo e progetti a gruppi

Nota: il titolo «solo Godot» e le classi descritte qui sotto sono quelli di inizio anno; oggi il corso riguarda 1INF, 2INF, 3INF e 4TI.

Contesto didattico

- **Docente:** Nicola. Sta imparando Godot in prima persona, in parallelo ai ragazzi.
- **Classi:** una **seconda** e una **terza** di **istituto professionale**. → Taglio **pratico**: fare prima, teoria in piccole dosi legata al fare; risultato visibile e "figo" a ogni passo; molto copia-modifica-sperimenta. Sfruttare al massimo i 4 livelli di aiuto dell'eserciziario.
- **Background dei ragazzi:** conoscono **Lazarus** (Free Pascal) a **livello base**. Hanno usato pochi componenti:
- **TButton** (il bottone)
- **TEdit** (la casella di testo)
- la proprietà **Caption** (il testo mostrato)
- Progetti fatti: cose molto semplici, tipo una **calcolatrice**.
- Quindi: **principianti**. Sanno cos'è un evento (click), una proprietà, una variabile; non hanno mai visto un "game loop" né la programmazione a oggetti avanzata.

Obiettivo del corso (quest'anno)

- Un corso che **diverta**: la priorità è tenerli **agganciati**.
- Tutti i progetti devono essere **molto usabili e accattivanti** — giochini che li attirano e che possono mostrare/giocare subito. Niente esercizi aridi.
- Difficoltà **graduale**, sempre con un risultato visibile a ogni passo.

Metodo (come spiegare)

- **Sempre in italiano**, semplice, **un passo alla volta**, con risultato a schermo a ogni tappa.
- **Bridge da Lazarus**: spiegare ogni concetto Godot partendo da ciò che già conoscono. Tabella di traduzione: | Lazarus (lo sanno) | Godot (nuovo) | |---|---| | Form | Scena (albero di nodi) | | Componenti (TButton, TEdit…) | Nodi (Button, LineEdit, Label…) | | Proprietà

(Caption…) nell'Object Inspector | Proprietà nell'Ispettore | | Gestore evento (`Button1Click`) | Segnale + funzione | | Object Pascal | GDScript (stile Python) |

- Concetto NUOVO chiave da introdurre bene: il **game loop** (`_process(delta)` gira ~60 volte/sec da solo). *"Lazarus reagisce, Godot pulsa."*

Progetti a gruppi (lavoro in team) — importante

Durante l'anno, ogni tanto, fare **progetti a gruppi di 2-4** ragazzi con i **ruoli divisi**, per insegnare a **sviluppare in team** (competenza lavorativa vera, spendibile): - uno sviluppa la **scena**; - uno i **nodi**; - uno l'**interfaccia grafica** (GUI); - uno i **movimenti**/logica di gioco.

Obiettivi: collaborazione, divisione dei compiti, **integrare il lavoro degli altri** (come in un vero team di sviluppo). Si lega bene a **Git**: ognuno lavora sul suo pezzo e poi si uniscono i contributi.

Attenzioni (dato il contesto): - Comporre i gruppi con cura; **ruoli chiari e a rotazione** così tutti provano tutto e nessuno resta indietro o si nasconde dietro i più bravi. - Ogni gruppo deve arrivare a una **piccola vittoria mostrabile** (coerente col motore "Vinci subito · Fallo tuo · Mostralo").

16.3 Struttura del corso e flusso Git (progetto per l'anno)

Struttura del corso e flusso Git

Due ambienti: "autore" (Nicola) vs "ragazzi"

Il corso stesso è versionato su Git, su due livelli: - **`main` = ambiente AUTORE (di Nicola).** La sua "cucina": prepara, avanza, migliora il corso; può essere anche in lavorazione/incompleto. - **Release per i RAGAZZI.** Quando una parte è pronta e testata si pubblica una **Release** taggata (`v1.0`, `v1.1`…): la versione **congelata e stabile** che i ragazzi ricevono. Non cambia sotto i loro piedi mentre Nicola lavora alle migliorie, e insegna davvero il concetto di **release/versione**. - **I ragazzi lavorano su una LORO copia** (fork o repo personale), **mai** sul master di Nicola: il suo resta intatto e ognuno ha il suo spazio per esercizi e quaderno. (Si sposa con la Fase 2: branch/PR.)

Ibrido a due fasi (la complessità Git cresce con loro) — SCELTA CONFERMATA

- **Fase 1 (inizio) — esercizi SEPARATI**, ognuno con la sua descrizione (scheda a 4 livelli). Git semplice: un **commit** per salvare la propria versione. Se sbagli un esercizio gli altri restano intatti (zero conseguenze, zero vergogna). In repo: cartella `esercizi/` con una sottocartella per esercizio.
- **Fase 2 (quando hanno confidenza / progetti a gruppi) — progetto che EVOLVE** con **branch → Pull Request → release**. Un gioco che cresce; ognuno sul suo branch, PR, merge, e release taggate (`v1.0` giocabile, `v1.1` con suoni…). In repo: cartella `progetto-gruppo/`.
- **Perché non partire dal flusso unico + PR:** per ragazzi che mollano facile un progetto unico che si rompe = frustrazione, e PR/merge all'inizio sono troppo. Prima farglielo **desiderare** (semplice), poi introdurlo quando il gioco di gruppo lo rende naturale.

16.4 Lezione, cruscotto e valutazione (testo completo delle regole 5, 13, 14, 15, 18, 22, 23, 24, 28, 29, 30, 32)

5. Libro di testo Classe 1 (workflow VINCOLANTE). Ogni lezione produce un documento **MD + PDF versionato** (e si **archivia anche l'HTML** quando la resa è da HTML). Poi Claude **aggiunge automaticamente** quel contenuto al **libro di testo complessivo** `classe-1/libro-di-testo/libro-classe1.md` (bump versione + voce nel changelog). Il **PDF serve per leggere/stampare**, l'**MD si dà ai ragazzi per la loro AI** (che spiega/traduce nella loro lingua). **Claude mette SEMPRE tutto su Git e lo tiene aggiornato** (MD, PDF e HTML).

13. Il bottone «I voti» è fisso, i KPI si concordano (14/10/2026, VINCOLANTE). Il voto parte sempre da 40 e sale man mano che i ragazzi fanno le cose; ogni riga spiega il perché (il conto). Cambia solo che cosa si misura (numero di PR, avanzamento nel percorso, …): nella tabella dei voti ci sono SEMPRE TUTTI I PRESENTI (REGOLA GENERALE, 14/10), anche chi non ha consegnato niente (40); gli assenti non compaiono. I KPI dipendono dalla lezione: si propongono a Nicola DOPO che ha visto il percorso proposto (mai prima) e si usano solo dopo il suo sì. **Ogni giorno (29/10/2026, Nicola: «ogni giorno dovresti propormi dei KPI, dei punti da monitorare, e io te li confermo o ne chiedo altri»):** appena è scelta la lezione, Claude propone in chat 2-4 KPI legati all'obiettivo del giorno (per esempio: «due versioni Lazarus su Git» = il codice giusto di ogni versione + le due versioni su GitHub), con il voto 40-80 che ne deriva; il cruscotto mostra per prime le colonne di quei KPI, una ✔/✘ per ragazzo, con una legenda «come si legge».

14. La lezione funziona da sola, senza il docente (14/10/2026, VINCOLANTE). Nicola in classe fa solo gli screenshot di Veyon, le note e dà indicazioni: non può controllare le pagine. Quindi Claude: valuta sempre lui (verifica e lavori), prepara il recupero individuale di ognuno, risponde alle domande dei ragazzi nella casella d'aiuto (Nicola solo in seconda battuta), tiene acceso il giro automatico che legge Modulo, aiuto e note e aggiorna backlog, cruscotto e voti. Nessun passo dei ragazzi deve dipendere da un controllo del docente. **Le domande dei ragazzi vanno all'assistente, non al prof (15/10/2026, Nicola):** lo scopo è scaricare il docente dalle domande, perché salga di livello: controlla l'andamento generale della lezione, aiuta di persona chi è bloccato e dà a Claude i feedback correttivi. Nelle pagine non si scrive «chiedi al prof» come strada normale: si chiede all'assistente; al prof si va solo per ciò che l'assistente non può fare (comportamento, salute, account bloccato, problemi del PC).

15. MAI PIÙ «copia, apri, incolla» per i ragazzi (14/10/2026, REGOLA ASSOLUTA di Nicola: «MAI PIÙ»). Nessuna pagina per i ragazzi chiede di copiare un testo, aprire un Modulo, incollare e premere Invia. Il ragazzo scrive il testo e preme UN bottone che consegna davvero (applicazione web dello script, oppure invio diretto al Modulo con i codici delle caselle). Se il canale diretto non c'è, la pagina NON si pubblica: lo si dice a Nicola prima della lezione. Vedi REGISTRO-ERRORI #81.

18. Il cruscotto dal vivo è lo standard di ogni lezione (19/10/2026). Regole fisse: ordine dal più bravo al meno (svolto all'inizio, voto alla fine), 🏁 per chi ha finito al controllo automatico, «manca: …» per gli altri, Veyon dentro il cruscotto, domande con risposta, voto proposto 40-80 con la regola delle domande, tutti i presenti, dati con nomi e voti sempre CIFRATI nel pubblico (`strumenti/cifra_json.py`). Vedi `CRUSCOTTO-DAL-VIVO.md`. **Logica fissa (21/10/2026, Nicola): il cruscotto serve a TENERE SOTTO CONTROLLO LE ATTIVITÀ DEGLI ALLIEVI.** È UNICO per la lezione: in una sola pagina tutte le attività del giorno (lettura della teoria, ripasso, esercizi, quiz su Moduli), una riga per ogni presente, chi è fermo, a che punto è, le attività rilevate da Veyon (fuori compito in rosso, quante volte) e le note della lezione (registro, uscite), voto proposto 40-80. **APPROVATO il 21/10/2026 («così va bene»): è il modello per tutte le lezioni.** Modello: `docs/avanzamento/oggi.html?t=<teoria>&e=`

`<esercizi>&q=<quiz cifrato>`. **Storico (21/10/2026, Nicola):** i dati dei cruscotti si storicizzano sempre, per poterli riguardare: a ogni giro una fotografia nel riservato (`storico/cruscotti/<classe>/<AAAA-MM-GG>/<HHMM>.json`); le consegne grezze restano in `dati/verifiche/<id>/arrivi/`. **Modello di default per la situazione quiz (29/10/2026, Nicola):** la pagina `<classe>-sicurezza-oggi.html` con una colonna per quiz (fino a 16 quiz, le 16 ore di Sicurezza della Regione; intestazioni corte 1ª…16ª, la tabella scorre di lato al suo interno con la colonna Allievo fissa), pastiglia verde/gialla, punteggio e data/ora. Generatore: riservato `generatori/situazione_quiz.py config/situazione-quiz/<id>.json`; pagina modello `docs/avanzamento/1inf-sicurezza-oggi.html` (legge `<x>-quiz.json`).

22. Report delle domande all'assistente, SEMPRE (26/10/2026, Nicola: «diventa una regola»). Durante ogni lezione controllo le domande che arrivano alla casella dell'assistente (`dati/verifiche/assistente-<pagina>/arrivi/` nel riservato), rispondo subito (`docs/assistente/risposte/<pagina>.json`, nome e iniziale) e do a Nicola in chat il report: ora, chi (in chat SEMPRE il nome anagrafico completo, COGNOME e nome, come nel registro: Nicola non riconosce i ragazzi dal solo nome o dal solo cognome; 26/10/2026), domanda, risposta data. Il report si dà a ogni giro, anche senza che lo chieda. **Le domande valgono solo in giornata (28/10/2026, Nicola):** si risponde lo stesso giorno; quelle rimaste senza risposta a fine giornata si chiudono e non si riprendono. **Pensare i ragazzi come dodicenni (26/10/2026, REGISTRO-ERRORI #123):** un passo = un'azione che si vede sullo schermo; niente cose da cercare sul PC se si possono fare su GitHub.

23. Quaderno della 1INF (28/10/2026, Nicola). Ai ragazzi di prima si dà sempre qualcosa da RICOPIARE sul quaderno (poche righe, dalla lavagna o dalla pagina): li attiva e li mette in silenzio. Claude lo prepara per ogni lezione della 1INF e lo ricorda a Nicola spesso, a ogni cambio di attività.

24. Report PDF per ogni ora, con gli screenshot (28/10/2026, Nicola: «NUOVA REGOLA»). Per ogni ora di lezione si crea un PDF di report con tutti gli screenshot di Veyon dell ora in risoluzione massima (originali, non ridotti), con ora, presenti, fuori compito, uscite, note e consegne. Contiene nomi: si salva SOLO nel repository riservato (report/AAAA-MM-GG/) e si manda a Nicola in chat.

28. Il cruscotto dice che cosa fa ogni ragazzo, al docente e ai ragazzi (28/10/2026, Nicola). Prima di ogni lezione Claude controlla che il cruscotto della classe mostri, per OGNI presente: a che punto è del lavoro del giorno (passo, scheda, quiz), che cosa ha consegnato su Classroom e che cosa c'è dentro (letto davvero), che cosa gli manca, da quanto è fermo, che cosa vede Veyon (fuori compito in rosso), note e uscite, voto proposto con il conto. La stessa informazione, senza note riservate e con Nome C., va nella pagina dei ragazzi linkata nel compito: si autoregolano. Se un dato manca, il cruscotto lo dice in rosso con l'ora dell'ultimo dato.

29. Tre modalità di lavoro e il cruscotto le segue tutte (28/10/2026, Nicola). Ogni lezione usa una o più di queste modalità: 1) lavoro con VERSIONING (GitHub: cartelle, file giusti, commit, release); 2) PERCORSO GUIDATO a passi controllati, con eventuale PERCORSO FACILITATO (versione corta, al massimo 3 passi) per chi è in difficoltà; 3) TEORIA a schede e QUIZ su Moduli. Il cruscotto del docente e la pagina dei ragazzi mostrano, per ogni presente, l'avanzamento in ciascuna modalità usata quel giorno e un'indicazione dettagliata del prossimo passo.

30. Claude risolve da solo; al docente solo le criticità (29/10/2026, Nicola: «Dammi bottone rosso per le criticità che necessitano mio intervento, per il resto prepara tutto, fai audit che funzioni, pubblica e risolvi tutti i casini da solo»). Claude prepara, verifica (audit), pubblica e risolve da solo. Al docente arrivano SOLO le criticità che richiedono il

suo intervento (firma del registro, presenze da mandare, PC rotto, ragazzo fuori dall'aula da più di 5 minuti, uno sblocco su Classroom o Apps Script che solo lui può fare, una decisione: scelta della proposta o «pubblica»): nel bottone rosso «Serve te (N)» del cruscotto (`docs/cruscotto/criticita.json`, scritto da Claude, cifrato con `strumenti/cifra_json.py` se ha nomi, #85; grigio «Niente per te» quando N = 0) e in chat con 🔴 in prima riga. Ogni voce: classe, ora, che cosa fare in una frase, bottone «Fatto» che arriva a Claude sul canale `docente-<classe>`. Ciò che Claude può fare da solo NON va nel bottone; una voce si toglie appena è risolta.

32. Ogni passo ha un'immagine che lo spiega (02/11/2026, Nicola: «sempre anche un'immagine che spieghi», «mettilo come regola»). In ogni pagina per i ragazzi, ogni passo operativo ha accanto un'immagine (screenshot reale se c'è, altrimenti schema disegnato) che mostra COSA si vede o COSA si clicca, con il punto cerchiato o la freccia. Il testo resta corto e l'immagine è la spiegazione principale: così capiscono anche gli allievi che leggono con fatica o in un'altra lingua. Un passo senza immagine non si pubblica, a meno che sia una sola frase senza azioni sullo schermo. Gli screenshot reali li manda Nicola; finché non ci sono, schemi in SVG, dichiarati come schemi.